

Corso di Crittografia

Prof. Dario Catalano

Primitive Asimmetriche

NON TUTTE SONO ATTUALMENTE IN USO

Introduzione

- Oggi discuteremo le primitive sulla base delle quali costruire sistemi asimmetrici affidabili.
- Nel caso della crittografia simmetrica tali primitive erano i cifrari a blocchi.
- Adesso guarderemo ad opportuni problemi tratti dalla teoria dei numeri computazionale.

$$\mathbb{Z}_m = \{0, 1, \dots, m-1\}$$

$$5 + 6 \bmod 11 = 0$$

Logaritmo discreto (su gruppi finiti)

PRIMO PROBLEMA CHE SFRUTTEREMO

- $|G|=m$ gruppo ciclico, g generatore
- $\text{DLog}_{G,g}: G \rightarrow \mathbb{Z}_m$

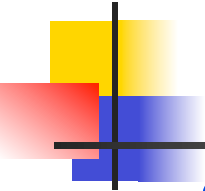
$$\text{DLog}_{G,g}(a)=x$$

$$g^x = a$$

- x e' tale che $g^x=a$ (in G)
- Operazione inversa all'op di esponenziazione.

QUESTA FUNZIONE
E' INTRATTABILE IN
QUESTO INSIEME
 $\mathbb{Z}_p^*$ NON E' OPPORTUNO

- Tale funzione e' congetturata essere unidirezionale (piu' dettagli in seguito)



$$\mathbb{Z}_B \{ 1, 2, \dots, 13 \}$$

$$4 \cdot 4 \bmod 13 = 3$$

IL MODULO FA SÌ CHE IL BINARY SEARCH NON SIA
ATTUABILE. Ad x molto piccolo possono corrispondere
a molto grandi e viceversa.

Il problema DL $\rightarrow$ FONDAMENTALE

- G gruppo ciclico di ordine m, g generatore di G.

$\text{Esp}_{G,g}^{\text{dl}}(A)$ *discrete log*
NOT ALL'AVVERSAIO

$x \leftarrow_R \mathbb{Z}_m; X \leftarrow g^x; x' \leftarrow_R A(X)$

If $(g^{x'} = X)$ Return 1 *RESTITUISCE IL SUO TENTATIVO*
else Return 0

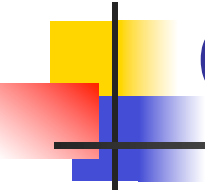
X grande e a in $\mathbb{Z}_{\text{ord}_g(A)}$ $\text{Adv}_{G,g}^{\text{dl}}(A) = \Pr[\text{Esp}_{G,g}^{\text{dl}}(A) = 1]$

SOLITO VANTAGGIO

È inattaccabile se $\text{Adv}(A)$ è $\approx$ per ogni mac. computazionale

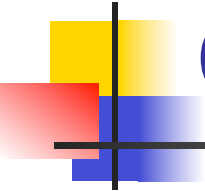
G g

$$g^x = h$$



Congettura

- Ipotizziamo che, per **determinati** gruppi, il problema del logaritmo discreto sia (asintoticamente) intrattabile.
 - Z_p^* , gruppi di punti di curve ellittiche su campi finiti
- Consideriamo il problema intrattabile purché si considerino gruppi di “taglia” sufficientemente grande.



Il problema Diffie-Hellman Computazionale

- Apparso per la prima volta nel contesto di scambio di chiavi.
- Alice e Bob vogliono scambiare una chiave comune senza incontrarsi.

NEE DIFFIE - HELLMAN

- CRYPTO ASIMMETRICO
- FIRM DI CRYPTO $\rightarrow$ MAC ASIMMETRICO
- SCAMBIO DI CHIAVI

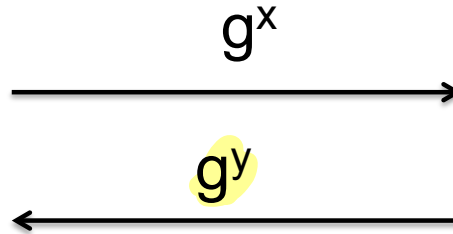
Il problema Diffie-Hellman Computazionale

Parametri comuni: G gruppo (di ordine m) nel quale il logaritmo discreto è (congetturato essere) difficile

Alice sceglie $x \leftarrow_R \mathbb{Z}_m$
e calcola g^x

Bob sceglie $y \leftarrow_R \mathbb{Z}_m$
e calcola g^y

x, g^y
 g^{xy}



g^x, y
 g^{xy}

Chiave condivisa: g^{xy}

$DL(g^x) \rightarrow x$
 $(g^y)^x$

INTRATTABILITÀ Δ DIFFIC. PELLMAN

Il problema CDH (cont.)

- G gruppo ciclico di ordine m, g generatore di G

QUESTO PROBLEMA È TALE PERCHÉ SI USA LOGARITMI DISCRETI

$\text{Esp}_{G,g}^{\text{cdh}}(A)$

$x, y \leftarrow_R \mathbb{Z}_m; X \leftarrow g^x; Y \leftarrow g^y;$

$Z \leftarrow_R A(X, Y)$

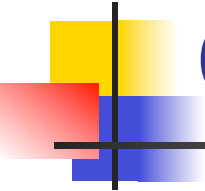
If $(Z = g^{xy})$ Return 1

else Return 0

$$\text{Adv}_{G,g}^{\text{cdh}}(A) = \Pr[\text{Esp}_{G,g}^{\text{cdh}}(A) = 1]$$

SOLITA COSA

Dimostrare che sono lo stesso problema. È ancora aperto

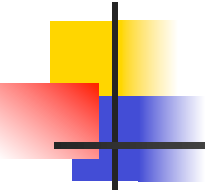


Osservazioni

- La funzione DH non può essere una funzione OW.
- È chiaro che il problema CDH non può essere più difficile del problema del logaritmo discreto.

$$\text{Adv}_{G,g}^{\text{dl}}(A) \leq \text{Adv}_{G,g}^{\text{cdh}}(A)$$

- Attualmente il solo modo che conosciamo per risolvere CDH è risolvere DL.

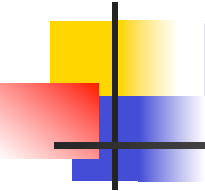

$$A^{CDH}(x, y) \rightarrow \vec{z} = g^{xy}$$

$$A^{DL}(x) = x \quad g^x = x$$

$$A^{CDH}(x, y)$$

$$\text{Run } A^{DL}(x) \rightarrow x$$

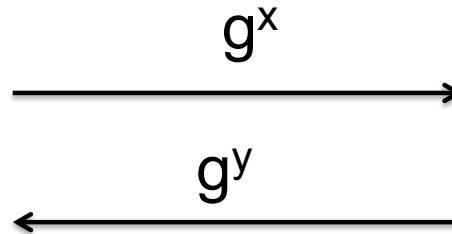
$$\text{OUTPUT } z = y^x$$



Protocollo Diffie-Hellman

Alice sceglie $x \leftarrow_R \mathbb{Z}_m$
e calcola g^x

Bob sceglie $y \leftarrow_R \mathbb{Z}_m$
e calcola g^y



Chiave condivisa: g^{xy} ← IMPREVEDIBILE
RANDOM

Problema: Chi ci garantisce che la chiave condivisa sia davvero random?

Il problema Diffie-Hellman Decisionale

G gruppo ciclico di ordine m , g generatore di G

$\text{Esp}_{G,g}^{\text{ddh-1}}(A)$ *DIFFIE-HELLMAN*

$x, y \leftarrow_R Z_m$; $z \leftarrow xy \bmod m$

$X \leftarrow g^x$; $Y \leftarrow g^y$; $Z \leftarrow g^z$

$d \leftarrow_R A(X, Y, Z)$

Return d

$\text{Esp}_{G,g}^{\text{ddh-0}}(A)$

$x, y, z \leftarrow_R Z_m$ *VERO CASO*

$X \leftarrow g^x$; $Y \leftarrow g^y$; $Z \leftarrow g^z$

$d \leftarrow_R A(X, Y, Z)$

Return d

$$\text{Adv}_{G,g}^{\text{ddh}}(A) = \Pr[\text{Esp}_{G,g}^{\text{ddh-1}}(A) = 1] - \Pr[\text{Esp}_{G,g}^{\text{ddh-0}}(A) = 1]$$

$$B^{CDH}(x, y) \rightarrow z$$

$$x = g^x \quad y = g^y \quad z = g^{xy}$$

$$Adv^{CDH}(B)$$

$$A^{DDH}(x, y, z)$$

$$\text{Run } B^{CDH}(x, y) \rightarrow \hat{z} \quad \rightarrow \text{LIMITED}$$

$$\text{IF } (\hat{z} == z) \text{ RETURN } 1$$

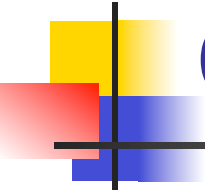
$$\text{ELSE RETURN } 0$$

$$Adv^{DDH}(A)$$

$$Pr[ESP^{DDH-1}(A) = 1] = Pr[ESP^{CDH}(B) = 1] = Adv^{CDH}(B)$$

$$Pr[ESP^{DDH-0}(A) = 1] = \frac{1}{|G|}$$

.

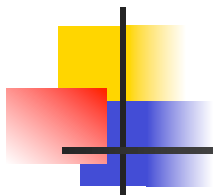


Osservazioni

- E' chiaro che il problema DDH non puo' essere piu' difficile del problema CDH.

$$\text{Adv}^{\text{cdh}}_{G,g}(A) \leq \text{Adv}^{\text{ddh}}_{G,g}(A) + 1/|G|$$

- In alcuni gruppi DDH e' facile (pur rimanendo CDH difficile)
- Se G e' scelto adeguatamente, il solo modo che conosciamo per risolvere DDH e' risolvere DL.



$$\mathbb{Z}_p^*$$

g

p primo ≥ 2

H non è intellabile

$$\mathbb{Z}_p^*$$

$p-1$

$$g^x$$

x

$$g^y$$

y

$$g^{xy \bmod p-1}$$

$xy \bmod m$

$$4 \cdot 2 \bmod 7$$

$$g^{2i}$$

$5 \cdot 7$

$\bmod 20$

$$35 = 1 \cdot 20 + 15$$

$$a \cdot b = q(2k) + r$$

A(X,Y,Z)

If $J_p(X)=1$ or $J_p(Y)=1$

set $s=1$

else set $s=-1$

If $J_p(Z)=s$ return 1

else return 0

s fissato esito $\frac{1}{2}$

VEDI ONE NOTE

$$g^x \quad g^y$$

$$g^{xy}$$

$$P_0[Exp^{ddh-1}(A)=1]=1$$

$$P_0[Exp^{ddh-0}(A)=1]=\frac{1}{2}$$

$$p-1 = kq$$

G

$$\hat{g} = g^{\frac{p-1}{k}}$$

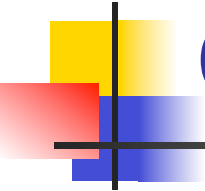
$$\hat{g}^q$$

n. y mod q

$$\hat{g}, \hat{g}^x, \hat{g}^y, \hat{g}^{xy \bmod q}$$

$$p = 2q + 1$$

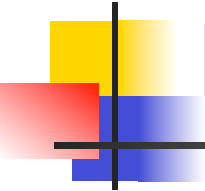
se rompo logaritmo discreto, rompo tutto il
sistema



Calcolare logaritmi discreti

I metodi migliori per calcolare logaritmi discreti sono essenzialmente di 2 tipi:

- **Metodi Index Calculus** → SPECIFICO
 - Piuttosto efficienti (ma **non polinomiali**). Richiedono la presenza di determinate proprietà aritmetiche per funzionare.
- **Metodi Collision Search** → GENERICO
 - Metodi generici, ma puramente esponenziali.



Logaritmi discreti in $\mathbb{Z}_p^*$

- Il problema DDH e' facile in questo gruppo
- L'algoritmo piu' efficiente in pratica per calcolare DL in $\mathbb{Z}_p^*$ e' GNFS (General Number Field Sieve)

$$O(e^{C+o(1)} \cdot \ln(p)^{1/3} \cdot (\ln \ln(p))^{2/3})$$

- Si tratta di un algoritmo euristico.

Algoritmi Generici per Calcolare logaritmi discreti

- Algoritmo Pohlig-Hellman

- Se la fattorizzazione di q (ordine del gruppo) è nota

- Baby-step Giant-Step (Shanks)

- In questo caso la complessità è

$$O((\sqrt{q}) \text{ polylog}(q))$$

COLLISION SEARCH
CLASSICO

$$|q| = m$$

$$2^{|q|} = 2^m \text{ passi}$$

$$\sqrt{q}$$

$$0 \leq B \leq 100$$

$$2^{\frac{m}{2}}$$

ONE REG

$$\boxed{a} + \boxed{b} \leq 10$$

Baby-Step / Giant-Step: idea

Input: g generatore di G , X elemento di G

Output: x tale che $g^x = X$

Supponiamo che $|G|=m$, $n = \lceil \sqrt{m} \rceil$

cerco due numeri e non tro

Allora possiamo porre $x = nx_1 + x_0$, con $0 \leq x_1, x_0 \leq n$

$$X = g^x$$

Per cui $X = g^{nx_1 + x_0} \Rightarrow Xg^{-x_0} = (g^n)^{x_1}$

$$X = g^{n_0 + n_1 u}$$

$$X g^{-n} = \underline{(g^u)^{n_1}}$$

$$X g^{-n} = (g^u)^{n_1}$$

$$\boxed{X g^{-n} = N^a}$$

Baby-Step / Giant-Step

Input: g generatore di G , X elemento di G

Output: x tale che $g^x = X$

Supponiamo che $|G|=m$, $n=\text{ceil}(\sqrt{m})$, $N=g^n$

1. For $b = 0$ to n $B[Xg^{-b}] \leftarrow b$ // baby step

2. For $a = 0$ to n // giant step $O(n)$
 $Y \leftarrow N^a;$

if $B[Y]$ is not empty $\rightarrow$ l'ho già trovato

$x_0 \leftarrow B[Y]; x_1 \leftarrow a;$

3. Return $nx_1 + x_0$

0	0	0	X	-	-	0	X	0	-	-	-	1
---	---	---	---	---	---	---	---	---	---	---	---	---

Lo SPAZIO SI PUÒ RIUTILIZZARE

Non deve fare nessuna ipotesi sulla
funzione.

Si può utilizzare per migliorare altri
algoritmi.

$$|Z_p^*| = p-1$$

Logaritmi discreti in Z_p^* (cont.)

- Se la fattorizzazione di $p-1$ e' nota si puo' fare ancora meglio.
- $p-1 = p_1^{a_1} \dots p_n^{a_n}$
- Si possono calcolare DL in tempo

$$\sum_{i=1}^n \underline{a_i} (\sqrt{p_i} + |p_i|)$$

$$p \quad 3000$$

$$p-1 \quad 2$$

$$P^{-1} = P_1 P_2 P_3$$

$$g, X \in \mathbb{Z}_p^*$$

$$g^x = X \pmod{p}$$

g generator

$$g^{P-1} = 1 \pmod{p}$$

$$\underline{g^{P_1 P_2 P_3} = 1 \pmod{p}}$$

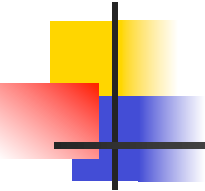
$$g^x = X \pmod{p}$$

$$(g^x)^{P_2 P_3} = X^{P_2 P_3}$$

$$g_1 = g^{P_2 P_3}$$

$$X_1 = X^{P_2 P_3}$$

$$\underline{g_1^x} = X_1 \pmod{p}$$



$$g_1^{p_1} = 1 \pmod{p}$$

$$g_1^{p_1} = (g^{p_2 p_3})^{p_1} = g^{p_1 p_2 p_3} = 1$$

$$g_1^x = x_1 \pmod{p} \quad O(p_1)$$

$$O(\sqrt{p_1}) \quad \boxed{x \pmod{p_1}}$$

$$x \pmod{p_1 p_2 p_3}$$

$$g^x = x \pmod{p}$$

$$(g^x)^{p_1 p_3} = x^{p_1 p_3} \pmod{p}$$

$$g_2 = g^{P_2 P_3}$$

$$x_2 = x^{P_2 P_3}$$

$$g_2^x = x_2 \bmod p$$

$$O(p_2)$$

$$x_2 \bmod p_2$$

$$O(\sqrt{p_2})$$

for $i = 1 \dots p_2 - 1$

if $g_2^i = x_2$ output i

$$(g^x)^{P_2 P_3} = (x)^{P_2 P_3} \bmod p$$

$$g_3 = g^{P_2 P_3}$$

$$x_3 = x^{P_2 P_3}$$

$$g_3^x = x_3 \pmod p \quad O(p_3) \quad n \pmod{p_3}$$

$$O(\sqrt{p_3})$$

$$n \pmod{p_1}$$

$$n \pmod{p_2}$$

$$n \pmod{p_3}$$

$$n \pmod{p_1 p_2 p_3}$$

$$n \pmod{p-1}$$

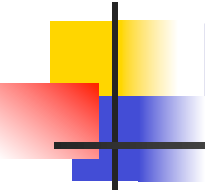
$$17$$

$$17 = 16 + 1$$

$$17-1 = 2^4$$

$$p = 2q+1$$

$$p-1 = 2q$$

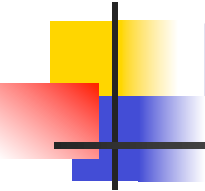


Esempio

- Sia $p-1=p_1p_2p_3$, g generatore di $\mathbb{Z}_p^*$.
- Dato $h=g^a \bmod p$, vogliamo calcolare a .

Idea:

- $h_1=h^{p_2p_3} \bmod p$, ha ordine p_1
- $h_2=h^{p_1p_3} \bmod p$, ha ordine p_2
- $h_3=h^{p_1p_2} \bmod p$, ha ordine p_3



Esempio (cont.)

- Sia $p-1=p_1p_2p_3$, g generatore di Z_p^* .
- Dato $h=g^a \bmod p$, vogliamo calcolare a .

Idea:

- $h_1=h^{p_2p_3} \bmod p=(g^{p_2p_3})^a \bmod p$
- $h_2=h^{p_1p_3} \bmod p=(g^{p_1p_3})^a \bmod p$
- $h_3=h^{p_1p_2} \bmod p=(g^{p_1p_2})^a \bmod p$
-

Il problema della fattorizzazione

- P^k insieme dei primi di taglia k .

$$f : P^k \times P^k \rightarrow N, f(p, q) = pq$$

- L' algoritmo piu' rapido conosciuto per fattorizzare e' il *Number Field Sieve*, superpolinomiale.
- La sua complessita' e' simile a quella di GNFS.
- Oggi fattorizzare interi di 1024 bit (meglio 2048) e' considerato intrattabile.

$$\mathbb{Z}_{N=55}^* = \{1, 2, 4, 7, 8, 10, 13, 14\}$$

$$RSA_{N,e} : \mathbb{Z}_N^* \rightarrow \mathbb{Z}_N^*$$

$$\mathbb{Z}_N^* = \{ 0 < x < N \wedge \gcd(x, N) = 1 \}$$

RSA

- Sia N il prodotto di due primi p, q ed e un esponente pubblico.
- La funz. RSA e' definita come

PERMUTAZIONE $\rightarrow$ $RSA[N,e](x) = x^e \bmod N$

- Primo es. di funzione trapdoor: dati e, N e $y = x^e \bmod N$, la conoscenza di p e q permette di ritrovare x

$$N = p_1^{\alpha_1} \dots p_t^{\alpha_t}$$

$$\varphi(N) = \prod_{i=1}^t (p_i^{\alpha_i} - p_i^{\alpha_i-1})$$

$$N=p$$

$$\varphi(N) = p-1$$

$$N=pq$$

$$\varphi(N) = (p-1)(q-1)$$

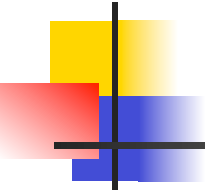
Dettagli di RSA

- $N=pq$, consideriamo il gruppo
 $Z_N^* = \{x \mid x < N ; \gcd(x, N) = 1\}$
 $Z_N = \{n \mid 0 \leq n < N\}$
- Esso ha ordine $\varphi(N)=(p-1)(q-1)$
- Dunque

$$\forall x \in Z_N^*, \quad x^{\phi(N)} \equiv 1 \pmod{N}$$

FATTORIZZAZIONE DI
 N

N=pq



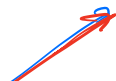
$$\text{RSA}[N, e] : \mathbb{Z}_N^* \rightarrow \mathbb{Z}_N^*$$

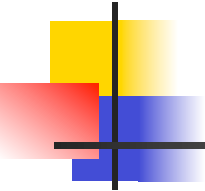
$$\text{RSA}[N, e](x) = x^e \bmod N = y \bmod N$$

$$\text{Inverti}(p, q, y) \rightarrow x$$

1. Conoscendo p, q , calcola $\phi(N) = (p-1)(q-1)$

2. Usando l'algoritmo esteso di Euclide
calcola d, f tali che

$$f \phi(N)$$




$$\boxed{\begin{aligned} de + f\phi(n) &= 1 \\ de &= 1 \pmod{\phi(n)} \end{aligned}}$$

$$de = 1 - f\phi(n)$$

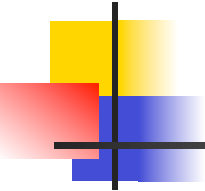
3. Abb d provides correct signature

$$y^d \pmod{N} = (n^e)^d \pmod{N} =$$

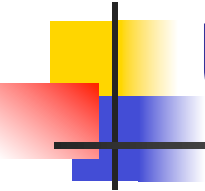
$$= n^{ed} \pmod{N} = n^{1-f\phi(n)} \pmod{N}$$

$$= n \cdot (n^{\phi(n)})^{-f} \pmod{N}$$

$$= n \pmod{N}$$


$$y = x^2 \text{ mod } N$$

$$\sqrt{y} \rightarrow x$$



Unidirezionalita' di RSA

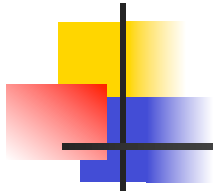
- Conoscere la fattorizzazione di N , permette di invertire RSA.
- E' questo l'unico modo in cui possiamo invertire RSA?

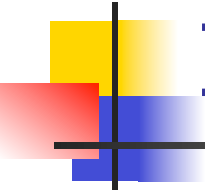
$$N_1 = p q_1$$

$$N_2 = p q_2$$

$$\gcd(N_1, N_2) = p$$

ATTACCHI $E_1, \bar{E}_2, \bar{E}_3$





Implementare RSA

- Fino ad ora abbiamo parlato di quanto e' difficile invertire RSA
- Per poterlo utilizzare dobbiamo pero' chiarire:
 - Possiamo generare (in modo efficiente) moduli RSA
 - Possiamo calcolare la funzione RSA in modo efficiente?

Sia $|N = pq| = 1024 \text{ bit} \Rightarrow p, q \approx 512 \text{ bit}$.

Domanda - Esistono abbastanza primi così grandi?

Sia $\pi(n)$ la **funzione di distribuzione dei primi** (numero di primi minori o uguali a n).

Teorema [Prime Number Theorem]

$$\lim_{n \rightarrow \infty} \frac{\pi(n)}{n / \ln n} = 1$$

L'approssimazione è accurata anche per n piccolo

Intuitivamente il teorema ci dice che un intero scelto a caso nell'intervallo $\{1, \dots, n\}$ è primo con prob. $1 / \ln n$.

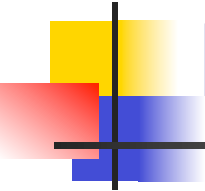
COME FACILIO A CAPIRE SE
HO UN NUMERO PRIMO?

CRIVELLO DI ERATOSTANE

NOT STONKS!

Test di primalita'

- Esistono diversi test di primalita' efficienti (probabilistici).
- Il primo test polinomiale e deterministico e' stato proposto nel Agosto 2002.
 - Algoritmo di interesse per lo piu' teorico
- Uno degli algoritmi piu' diffusi e' **Miller-Rabin** → PROBABILISTICO, PUO' DARE UNA RISPOSTA SBAGLIATA (BASSO)



Miller-Rabin

- Idea: Su input un intero a l'algoritmo cerca una “prova” che a è composito.
- Se il test restituisce Composito la risposta è corretta con probabilità 1.
- Se restituisce Primo la risposta è sbagliata con probabilità 2^{-2s} .
- Il test ha complessità $O(s|a|^3)$.



CUBICO

$$23 =$$

Miller-Rabin(p)

$$23 = 2 \cdot 11 + 1$$

$$17 = 2^4 \cdot 1 + 1$$

FAILE DA CALCOLARE

Sia $p = 2^k m + 1$ (m dispari)

a (random) t.c. $1 < a \leq p-1$

$$p-1 = 2^k m$$

TOTALMENTE
INDIPENDENTE

$b = a^m \bmod p$

If $b == 1 \bmod p$ **then** return "prime"

For $i=0$ **to** $k-1 \rightarrow b = a^{2^{k-i} m}$

If $b == -1 \bmod p$ **then** return "prime"

else $b = b^2 \bmod p$

Return "composite"

Supp. de p sia primo

NON VEDIAMO SE p NON È PRIMO

$$a^{p-1} = 1 \pmod{p}$$

$$p-1 = 2^k m$$

ITERAZIONE $k-1$
SEZ CICLO

$$a^{2^k m} = 1 \quad m > 1 \quad p$$

SPACIO LA RADICE QUADRATA

DIREBBE PRIMO

$$a^{2^{k-1} m} = -1$$

$$a^{2^{k-2} m} = 1$$



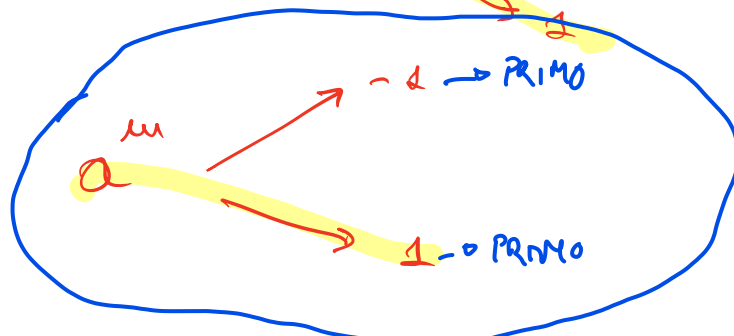
$$a^{2^{k-2} m} = -1$$

$$a^{2^{k-3} m} = 1$$

CONTINUO

$$a^{2^m} = 1$$

LE RADICI



GENERO SOLO YISARI

FA PROVO A FARE IL MODULO PER VEDERE SE QUEL NUMERO

Miller-Rabin

È MULTIPOLO DI PRIMI PICCOLI. SE

PASSA TUTTI I TEST, ALLORA PROVO

MILLER RABIN

- Se p è primo l'algoritmo restituisce "composito" con prob. 0
 - Analisi semplice
- Se p è composito la probabilità di ottenere "primo" come risposta è non nulla
 - Analisi più complicata

COME GENERO UN PRIMO FORTE?
 $p = 2q + 1$ GENERO q PRIMO COME VISTO PRIMA

È Poi SPERO.

N e, $\phi(w)$

(N, e)

$n^e \text{ mod } W$

n^5

$e \cdot d \equiv 1 \text{ mod } \phi(w)$

n^e

$\text{Exp}(x, e)$

if $e == 0$ output 1

else if $e == 1$ output x

else $x \cdot \text{Exp}(x, e-1)$

CERCHIAMO $O(e)$ TEMPO $O(e)$ SPAZIO
SI OTTIMIZZARE

$$n^e = \begin{cases} 1 & \text{if } e == 0 \\ \left(n^{\frac{e}{2}}\right)^2 & \text{if } e \text{ pari} \\ n \left(n^{\frac{e-1}{2}}\right)^2 & \text{if } e \text{ dispari} \end{cases}$$

SEMPRE QUALCOSA CHE È LA METÀ DI PRIMA

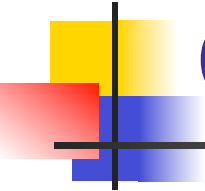
$\text{Exp}(n, e)$

if $e == 0$ output x

if e pari output $(\text{Exp}(n, \frac{e}{2}))^2$

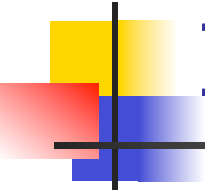
if e dispari output $x \times (\text{Exp}(x, \frac{e-1}{2}))^2$

$O(\log e)$



Calcolare la funz. RSA

- Dobbiamo calcolare $x^e \bmod N$
- In linea di massima sia x che e potrebbero essere elementi di (circa) $|\varphi(N)|$ bit.
- Il problema e' come fare tale operazione in modo efficiente.



Il metodo Square and Multiply

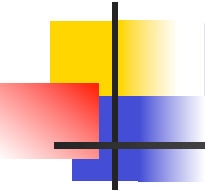
Idea: Calcolare $x^e \bmod N$ sfruttando la rappresentaz. binaria di e .

Sia $e_k e_{k-1} \dots e_0$ la rapprr. di e .

$$e = e_k 2^k + \dots + e_1 2 + e_0$$

Dunque

$$x^e \bmod N = \underbrace{\left(x^{2^k}\right)^{e_k}} \cdot \underbrace{\left(x^{2^{k-1}}\right)^{e_{k-1}}} \cdots \underbrace{\left(x^2\right)^{e_1}} \cdot \underbrace{x^{e_0}} \bmod N$$



L'algoritmo

$$x^e \bmod N = \left(x^{2^k}\right)^{e_k} \cdot \left(x^{2^{k-1}}\right)^{e_{k-1}} \cdots \left(x^2\right)^{e_1} \cdot x^{e_0} \bmod N$$

1. $d = 1$
2. **for** $i = k$ **downto** 0
3. $d = d \cdot d \bmod N$
4. **if** $e_i = 1$
5. $d = d \cdot x \bmod N$
6. **return** d

so quando
il bit è 0
e quando è
1

C'è un attacco basato sulla scomposizione del
tempo di calcolo della ~~CPU~~ $m^{SK} \bmod N$ $\rightarrow$ FRAMME QUESTO
POSSO ESTRARRE
L'ESPOLENTE

ESSENDO ALGORITMI

CONSTANT-TIME
PER CONTRARIARE
L'ATTACCO

m ~~no~~

~~SK~~ ~~secret~~

CHIARE IN CASO

x ~~SK~~

$$x' \leftarrow r \cdot x \bmod N$$

$$\boxed{x}^{SK}$$

$$x' / r^{SK} \rightarrow x^{SK}$$